



Shell Scripting



Overview

- An Introduction to Shell Scripting
- The How and Why Shell Scripting is Used
- Variables and Environment Variables
- Arguments and Expressions
- Types of Operators
- Conditionals and Loops
- Running a Shell Script

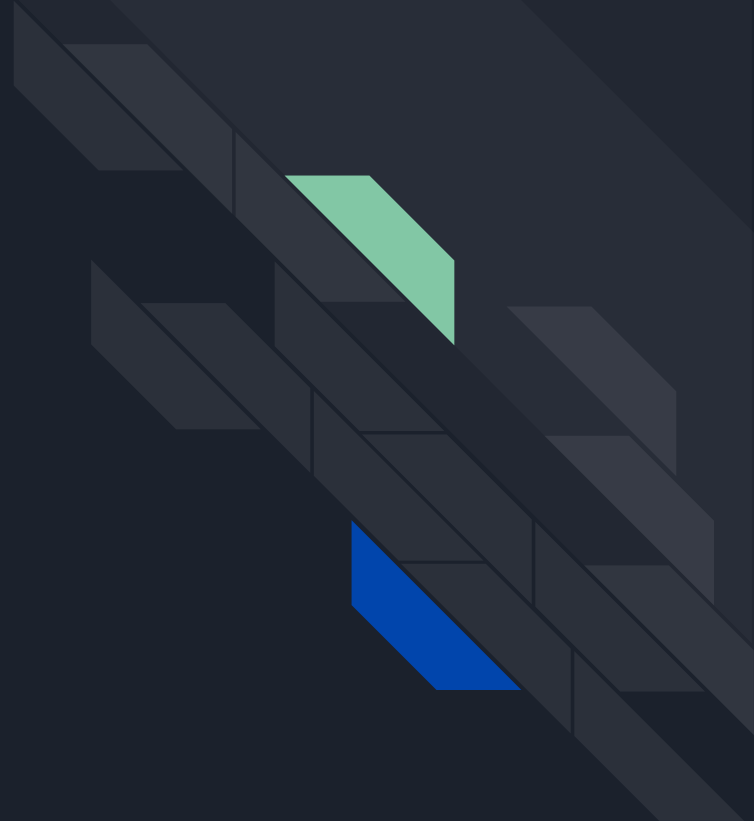


Learning Objectives

We will be able to:

- Describe what shell scripting is used for, and why we might choose it.
- Practice running scripts in a shell
- See how Arguments and Variables work with scripts
- Describe which conditionals and loops can be used in shell scripts
- Write our own shell script

Before We Begin... Let
Me Welcome You To....





What is a Shell? (not the dino kind!)

- The Shell is your command center
- It's like talking to the park's control room
- The shell interprets your commands
- There are different shells
- You can automate tasks





What is Bash?

- Bash Stands for “Bourne Again Shell”
- Bash is the shell you will use most often - it is the default shell for Linux operating systems!
- It's like your control console in the park
- Bash can run scripts
- Used for everything from setting up systems to launching dino enclosures

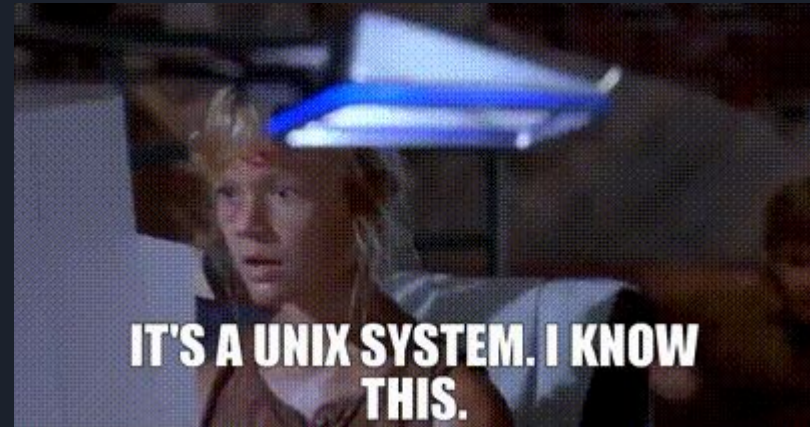
So, What is Shell Scripting?

- Incredibly powerful — and widely used by developers
- Also called Bash scripting
- Remember those awesome commands? `cat`, `sed`, `|`, `>`, `echo`, `ls` — all your old friends.
- Shell scripting = Writing a list of commands
- Think of it like giving the system instructions on autopilot



What is Shell Scripting Used for? Hint: Way more than opening dinosaur gates

- Configuring applications
- Deployments
- Installing packages
- Automating repetitive tasks
- Basically: anything you can type into the terminal, you can script.



Why Shell Scripts? (Because sometimes, you just want to open the gate without building the whole park)

- Shell scripts can do what other languages can do...
- Available anywhere on any Linux machine
- No dependencies = Guaranteed compatibility
- Use the powerful Unix commands
- Quick & simple for basic tasks





An Example of a Shell Script

```
1  #!/bin/bash
2
3  echo "Welcome to Jurassic Park"
4  sleep 1
5
6  echo -n "Please enter your name, park visitor: "
7  read name
8  sleep 1
9
10 echo "Scanning ID badge for $name..."
11 sleep 2
12
13 echo "Identity confirmed. Access level: Visitor"
14 sleep 1
15
16 echo "Assigning you a dinosaur companion..."
17 sleep 2
18
19 # Simple array of dinosaurs
20 dinos=("Triceratops" "Velociraptor" "T-Rex" "Brachiosaurus" "Pteranodon" "Stegosaurus")
21
22 chosen_dino=${dinos[$RANDOM % ${#dinos[@]}]}
23 echo "Your assigned dinosaur is: $chosen_dino!"
```



How to do shell scripting?

- Save your script commands to a file
- Give your script execution permissions (chmod)
- Run `chmod +x script.sh`
- Run the script with `./script.sh`
- `.sh` file extension

Change Mode (chmod)

```
chmod permissions filename
```

rwX	rwX	rwX
user	group	other

Regular file
-rw-r--r--
drwxr-xr-x
A directory



Parts of a Script: Shebang #!

You will often see `#!/bin/bash` or `#!/usr/bin/bash` at the start of shell scripts.

This is the 'Shebang' - It tells the shell what interpreter to run the script with.

Without the shebang we would have to run our scripts explicitly with `bash` e.g. `bash my_script.sh`.

With it we can run them with `./my_script.sh`.

You can add `#!/usr/bin/env python3` to your python application to execute it without writing the `python3` command every time.

```
1  #!/bin/bash ←
2
3  # This is a basic script to show how shebang works
4
5  echo "Welcome to Shellassic Park!"
6  DINOSAUR="T-Rex"
7  echo "Our star attraction today is the $DINOSAUR."
8
9  # Trying a simple command to show shebang in action
10 DATE=$(date)
11 echo "The current date and time is: $DATE"
```



Parts of a Script: Variables (the DNA of your Scripts!)

- Same concept as Python! Just a different syntax!
- A variable stores a value
- By default, variables are strings
- Variables = Labels for Memory
- Call on variables to substitute values in commands
- Think of it as a way to store & reuse data

```
timeout=120
```

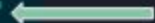
```
$<variable_name>
```



Shell Syntax for Variables

- Prefix with \$ to reference the value
- No spaces in variable names
- Can contain letters, numbers, or underscores
- Typically capitalized

```
1  #!/bin/bash
2
3  # This is a basic script to show how shebang works
4
5  echo "Welcome to Shellassic Park!"
6  DINOSAUR="T-Rex"
7  echo "Our star attraction today is the $DINOSAUR."
8
9  # Trying a simple command to show shebang in action
10 DATE=$(date)
11 echo "The current date and time is: $DATE"
```





Using Shell Variables on the Command Line

```
AzureAD+JessicaSnead@DESKTOP-C3EDFSP MINGW64 ~/OneDrive - Generation/Documents/data-engineering/de-modules/Generation-DE-Modules-Jess/shell-scripting/examples (main)
```

```
$ DINOSAUR_NAME="Triceratops"
```

```
AzureAD+JessicaSnead@DESKTOP-C3EDFSP MINGW64 ~/OneDrive - Generation/Documents/data-engineering/de-modules/Generation-DE-Modules-Jess/shell-scripting/examples (main)
```

```
$ DINOSAUR_AGE=70
```

```
AzureAD+JessicaSnead@DESKTOP-C3EDFSP MINGW64 ~/OneDrive - Generation/Documents/data-engineering/de-modules/Generation-DE-Modules-Jess/shell-scripting/examples (main)
```

```
$ echo "The mighty $DINOSAUR_NAME is $DINOSAUR_AGE million years old."
```

```
The mighty Triceratops is 70 million years old.
```

```
AzureAD+JessicaSnead@DESKTOP-C3EDFSP MINGW64 ~/OneDrive - Generation/Documents/data-engineering/de-modules/Generation-DE-Modules-Jess/shell-scripting/examples (main)
```

```
$
```



Using Shell Variables in a Script

```
1  #!/bin/bash
2
3  # Assign values to variables
4  DINOSAUR_NAME="T-Rex"
5  DINOSAUR_AGE=65
6
7  # Use variables in commands
8  echo "The mighty $DINOSAUR_NAME roamed the earth $DINOSAUR_AGE million years ago."
```



Environmental Variables

- Environment variables are a different type of variable that is inherited inside a script from the outside environment in which that script runs.
- We do not need to explicitly pass environment variables into a script like arguments, they are available automatically.

- Environment variables are created by various sources:
 - The operating system itself
 - Your ~/.bashrc or ~/.zshrc where custom environment variables are set
 - Other scripts or programs which may set new environment variables



Environmental Variables Examples

Environment variables are UPPERCASE

`$SHELL` - What shell you are using.

`$USER` - Who you're logged in as.

`$HOME` - Path to your home directory.

You can see other environment variables by using the `env` command.

You should not change the value of existing environment variables (without good reason) because it might break things that depend on those values.



Double vs Single Quotes

Strings wrapped in single quotes are used exactly as they appear by the shell.

String wrapped in double quotes will have variable names replaced with the value of that variable.

```
GNU nano 8.2 quotes.sh
main_character=Alan
secondary_character=Dennis

echo "The main character in Jurassic Park is $main_character"
echo 'A secondary character in Jurassic Park is $secondary_character'
```

```
AzureAD+JessicaSnead@DESKTOP-C3EDFSP MINGW64 ~/OneDrive - Generation/Documents/data-engineering
/de-modules/Generation-DE-Modules-Jess/shell-scripting/examples (main)
$ ./quotes.sh
The main character in Jurassic Park is Alan
A secondary character in Jurassic Park is $secondary_character
```



Emoji Check!

Do you feel you understand the use of environment variables and single vs double quotes?

- 😓 Haven't a clue, please help!
- 😞 I'm starting to get it but need to go over some of it please
- 😐 Ok. With a bit of help and practice, yes
- 😊 Yes, with team collaboration could try it
- 😁 Yes, enough to start working on it collaboratively



Exercise Time!



Parts of a Script: Options

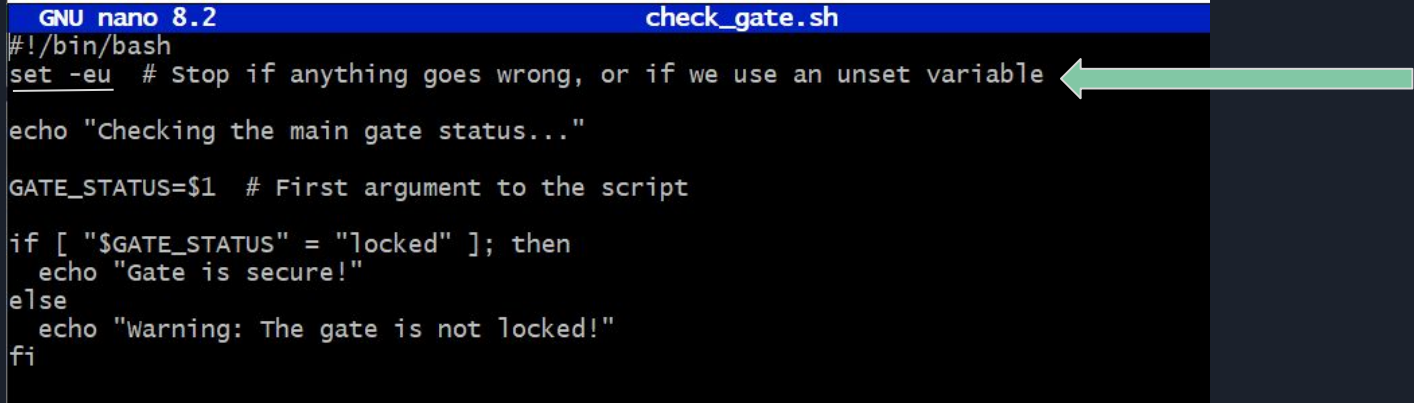
Options control how the shell will run the script.

These can be set by using `set` in your script. Common options you might use:

- e: Exit immediately if a command fails

It's best practice to always fail a script the moment a command fails

- u: Treat unset variables and parameters as an error



```
GNU nano 8.2 check_gate.sh
#!/bin/bash
set -eu # Stop if anything goes wrong, or if we use an unset variable

echo "Checking the main gate status..."

GATE_STATUS=$1 # First argument to the script

if [ "$GATE_STATUS" = "locked" ]; then
    echo "Gate is secure!"
else
    echo "warning: The gate is not locked!"
fi
```

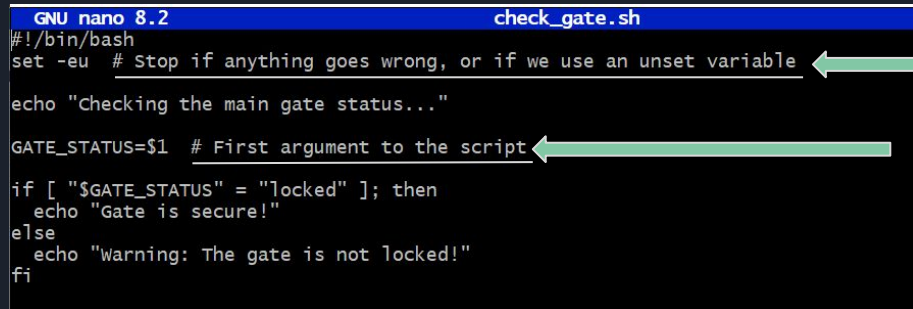

Parts of a Script: Comments

Anything on a line after # will be commented out.

Don't forget to comment your scripts. It's good practice to write what the script does at the top after the shebang.

Try to include a help command, and explain how to invoke the script by giving an example

Comments # are different from the shebang #!



```
GNU nano 8.2 check_gate.sh
#!/bin/bash
set -eu # Stop if anything goes wrong, or if we use an unset variable
echo "Checking the main gate status..."
GATE_STATUS=$1 # First argument to the script
if [ "$GATE_STATUS" = "locked" ]; then
    echo "Gate is secure!"
else
    echo "Warning: The gate is not locked!"
fi
```

Parts of a Script: User Input (read)

The read command lets your script ask the user for input during runtime.

It pauses the script and waits for the user to type something — then stores that input in a variable you can use later.

Pretty much input() from Python!



```
GNU nano 8.2 park_checkin.sh
#!/bin/bash
set -eu

echo "Welcome to Shellassic Park Security Check-In"

read -p "what's your name, Ranger? " RANGER_NAME
read -p "which dinosaur pen are you visiting today? " DINO_TYPE

echo "Hello $RANGER_NAME! Logging your visit to the $DINO_TYPE pen..."
sleep 1
echo "Visit recorded. Stay safe out there!"
```

Parts of a Script: Arguments

`$<index of argument>`

Values can be passed into scripts from outside the script with the use of arguments.

Arguments are assigned an index number (e.g. 1, 2, 3) based on the order they appear.

Arguments can be accessed by using that index number with `$`

```
GNU nano 8.2      check_gate.sh
#!/bin/bash
set -eu # Stop if anything goes wrong, or if we use an unset variable

echo "Checking the main gate status..."

GATE_STATUS=$1 # First argument to the script
if [ "$GATE_STATUS" = "locked" ]; then
    echo "Gate is secure!"
else
    echo "Warning: The gate is not locked!"
fi
```

```
GNU nano 8.2      ranger_greeting.sh
#!/bin/bash

# $1 is the first argument: the ranger's name
# $2 is the second argument: the dinosaur type

RANGER_NAME=$1
DINO_TYPE=$2

echo "Welcome, Ranger $RANGER_NAME!"
echo "You are assigned to the $DINO_TYPE enclosure today."
echo "Make sure to do the morning checks and log any suspicious dinosaur behavior."
```



Emoji Check!

How do you feel about using a script with a shebang and arguments and variables so far?

- 😓 Haven't a clue, please help!
- 😞 I'm starting to get it but need to go over some of it please
- 😐 Ok. With a bit of help and practice, yes
- 😊 Yes, with team collaboration could try it
- 😁 Yes, enough to start working on it collaboratively

Exercise Time!





Parts of a Script: Conditionals

Again, very similar to Python!

The if condition is inside square brackets `[[ ]]` and must evaluate to true or false.

An if condition is ended by `fi` (which is if backwards!):

```
GNU nano 8.2      check_gate.sh
#!/bin/bash
set -eu # Stop if anything goes wrong, or if we use an unset variable

echo "Checking the main gate status..."

GATE_STATUS=$1 # First argument to the script

if [ "$GATE_STATUS" = "locked" ]; then
    echo "Gate is secure!"
else
    echo "Warning: The gate is not locked!"
fi
```

```
GNU nano 8.2      check_gatep2.sh
#!/bin/bash

echo "Checking the main gate status..."

GATE_STATUS=$1

if [ "$GATE_STATUS" = "locked" ]; then
    echo "Gate is secure!"
elif [ "$GATE_STATUS" = "open" ]; then
    echo "Gate is open, alerting security."
else
    echo "Unknown gate status!"
fi
```



Numerical Operators

These are used with number-type variables

-gt : Greater than

-ge : Greater than or equal

-eq : Equals

-ne : Not Equal

-lt : Less than

-le : Less than or equal



String Conditional Operators

These are used with string-type variables

= or == : Strings are equal

!= : Strings are not equal

-z : String has zero length

-n or no operator : String has non-zero length



File Conditional Operators

These are used with files

- `-e` or `-a` : File exists
- `-f` : File exists and is regular file
- `-d` : File exists and is directory
- `-s` : File exists and is not empty
- `-r` : File is readable
- `-w` : File is writable
- `-x` : File is executable

```
#!/bin/bash
set -eu # Exit on any error or unset variable

# Prompt the user to enter a filename
echo -n "Enter the filename: "
read filename

# Check if the file exists and is a regular file
if [[ -f "$filename" ]]; then
    echo "$filename exists!"
else
    echo "$filename does not exist or is not a regular file."
fi
```



Emoji Check!

How do you feel about using bash variables?

1. 😭 Haven't a clue, please help!
2. 😞 I'm starting to get it but need to go over some of it please
3. 😐 Ok. With a bit of help and practice, yes
4. 😊 Yes, with team collaboration could try it
5. 😄 Yes, enough to start working on it collaboratively



Exercise Time!





Parts of a Script: For Loops

Can be defined as specific values or as a range.

Must include a done to denote when the loop ends.

What do you think is happening here?

for VARIABLE in VALUES; do

code to be executed

done

```
GNU nano 8.2 for_loop.sh
#!/bin/bash

# Loop through numbers 1 to 5
for NUMBER in 1 2 3 4 5; do
    echo "This is number $NUMBER"
done
```



Parts of a Script: While Loop

Much like python, will loop until condition evaluates to false.

```
while [CONDITION]; do
```

```
# Code to be executed
```

```
done
```

```
GNU nano 8.2                                dino_count.sh
#!/bin/bash

# Initialize the dinosaur count
DINOSAUR_COUNT=1

# while the dinosaur count is less than or equal to 5
while [ $DINOSAUR_COUNT -le 5 ]; do
    echo "We have $DINOSAUR_COUNT dinosaurs in the park."
    # Add one more dinosaur
    DINOSAUR_COUNT=$((DINOSAUR_COUNT + 1))
done

echo "The park now has $DINOSAUR_COUNT dinosaurs!"
```



Emoji Check!

How do you feel about using conditionals and loops in bash?

1. 😭 Haven't a clue, please help!
2. 😞 I'm starting to get it but need to go over some of it please
3. 😊 Ok. With a bit of help and practice, yes
4. 😊 Yes, with team collaboration could try it
5. 😊 Yes, enough to start working on it collaboratively



Exercise Time!



You did it. You crazy script writer, you did it.





Quick Recap!

Shell Script: A computer program to automate repetitive tasks.

Shebang or Hashbang: This tells the shell what interpreter to use.

Shell options: Settings that change shell and/or script behaviour.

Environment Variables: Variables that our applications inherit from their parent scope.



Scripting CheatSheet

<https://devhints.io/bash>



Some Other Fun Resources:

<https://tldp.org/LDP/abs/html/part1.html>

<https://www.youtube.com/watch?v=v-F3YLd6oMw>

<https://www.howtogeek.com/67469/the-beginners-guide-to-shell-scripting-the-basics/>



Now Some Practice!